High-Load Ticket System: План разработки

Пошаговый трек для перехода на микросервисы, Docker, Redis, Kafka и Nginx

Этот план разработан специально для постепенного погружения в production-инфраструктуру. Мы будем двигаться от простых вещей к сложным, разбирая каждую технологию с нуля прямо по ходу дела. Не бойся незнакомых слов — мы разберем всё построчно.

Этап 1: Подготовка окружения и первый микросервис

Шаг 1

Установка Docker и создание docker-compose

На этом шаге мы не будем глубоко учить Docker. Твоя задача — просто запустить готовые контейнеры баз данных и очередей одной командой, чтобы не ставить их к себе на компьютер.

- Установка Docker Desktop на компьютер.
- Создание файла `docker-compose.yml` в корне проекта.
- Прописывание образов для **PostgreSQL** и **Redis**.
- Запуск команды `docker-compose up -d` и проверка, что базы поднялись.

Docker PostgreSQL Redis

Шаг 2

Выделение Auth-Service (TypeScript + Чистая архитектура)

Берем твою текущую заготовку с логином/паролем и превращаем её в изолированный первый микросервис.

- Создание папки `services/auth-service`.
- Настройка `tsconfig.json`, `package.json`, Express и связи с PostgreSQL в Docker.
- Реализация хэширования паролей (с использованием **bcrypt**, как мы обсуждали).
- Генерация и выдача JWT-токенов (Access / Refresh) при успешном входе.

Node.js TypeScript JWT

Этап 2: Маршрутизация и Шлюз

Шаг 3

Внедрение Nginx как API Gateway

Теперь нужно сделать так, чтобы клиент не стучался в микросервисы напрямую по разным портам, а общался только с одним главным сервером.

- Добавление сервиса **Nginx** в наш обшний `docker-compose.yml`.
- Написание файла конфигурации `nginx.conf`.
- Настройка Reverse Proxy: перенаправление всех запросов с `localhost/api/auth/*` внутрь контейнера с Auth-Service.
- Проверка авторизации через Nginx с помощью Postman.

`Nginx` `Reverse Proxy`

Этап 3: Работа с высокой нагрузкой (Redis)

Шаг 4

Создание Ticket-Service и интеграция Redis

Создаем второй микросервис, который будет отвечать за создание концертов и бронирование мест.

- Создание папки `services/ticket-service`.
- Реализация эндпоинта `POST /tickets/reserve` (забронировать место на 10 минут).
- **Подключение Redis:** вместо записи в Postgres, мы пишем статус места "забронировано" в Redis с использованием механизма **TTL (Time To Live)** на 600 секунд.
- Если за 10 минут не произойдет оплата, Redis сам удалит этот ключ, и место снова станет доступным для других.

`Redis Cache` `TTL` `Microservices`

> 💡 **Чему ты научишься на этом этапе:** Ты поймешь, как Redis спасает основную базу данных от миллионов быстрых запросов на чтение и запись при старте продаж билетов.

Этап 4: Асинхронное взаимодействие (Kafka)

Шаг 5

Подключение Apache Kafka и создание Order-Service

Самый мощный этап. Сервисы больше не будут вызывать друг друга через HTTP. Они будут общаться через сообщения.

- Добавление **Kafka** и **Zookeeper** в `docker-compose.yml`.
- Создание третьего микросервиса: `services/order-service` (обработка заказов и фейковая оплата).
- Внедрение библиотеки `kafkajs` в наши сервисы.
- Когда в Ticket-Service создается бронь, он публикует событие в топик Kafka `order-created`.
- Order-Service асинхронно слушает этот топик, забирает заказ и проводит оплату, после чего кидает событие `order-paid` обратно в Кафку.

Apache Kafka | Event-Driven

Этап 5: Продвинутая безопасность и Полировка

Шаг 6

Интеграция OpenID Connect / OAuth 2.0

Делаем авторизацию профессиональной, защищая наши микросервисы от неавторизованных пользователей.

- Настройка мидлвара валидации JWT на уровне API Gateway (Nginx) или общего базового пакета.
- Изучение концепции OpenID Connect для возможности безопасного шеринга профиля пользователя между сервисами.
- Наведение порядка в коде, написание красивого README.md файла для GitHub с архитектурной схемой проекта.

OAuth 2.0 | OpenID Connect | Security